

ZK-Battleships

Orestis Alpos

Dominik Apel

Common Prefix
Engineering on-site, September 2025
Athens, Greece





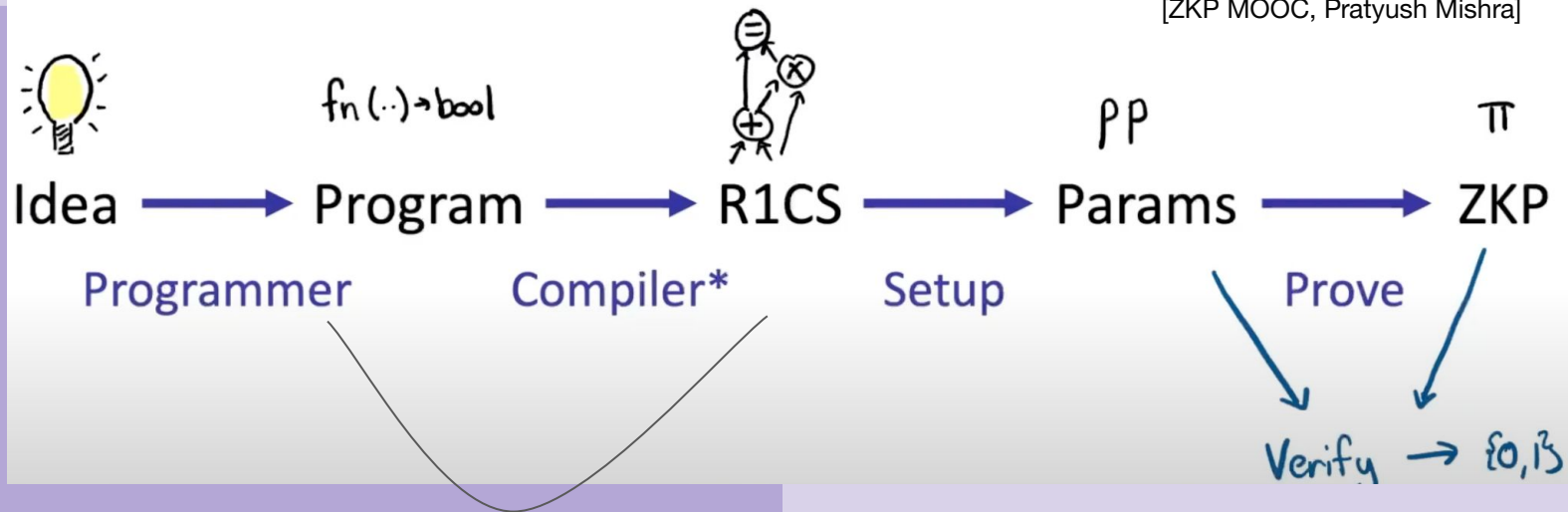
zkSNARK Basics



Zero-Knowledge Succinct Non-Interactive Argument of Knowledge

- Argument? Allow for soundness error
- Succinct? Proof size + verify time are **constant**
- Non-Interactive? Allow to e.g. post proofs on chain
- Argument of Knowledge? Can be sure that prover **knows** witness

[ZKP MOOC, Pratyush Mishra]



- Hardware-Description Language (e.g., Circom)
- “Standard” PL + Compiler
- Library in “standard” language (e.g, arkworks)
- Domain-Specific Languages (ZoKrates)



ZoKrates Basics



Data types

- `field: integer in [0, p - 1]`
- `bool`
- `u8, u16, u32, u64`
- `arrays`
- `structs`

Data types

- `field: integer in [0, p - 1]`
- `bool`
- `u8, u16, u32, u64`
- `arrays`
- `structs`

```
field[3] a = [1, 2, 3];  
bool[13] b = [false; 13];
```

```
struct Point {  
    field x;  
    field y;  
}
```

Conditionals

```
field x =  
if cond {  
    LB  
}  
else {  
    RB  
}
```

equivalent to

$$x = LB * cond + RB (1-cond)$$

where

- `cond` is a bool
- `cond`, `LB`, `RB` cannot have side effects

Statement and witness

- Add statement as inputs to main function
- Add witness as private inputs to main function



Example



A trivial one

```
def main(private bool b){  
  assert(if b {true} else {false});  
}
```

```
# curve:          bn128  
# constraint_count: 2  
def main(private _0) -> () {  
  (1 * _0) * (1 * _0) == 1 * _0 // Argument bitness check failed  
  (1 * ~one) * (1 * ~one) == 1 * _0 // Assertion failed at playground.zok:6:5  
  return  
}
```

- $_0 \rightarrow b$ (private input)
- $\sim\text{one} \rightarrow 1$ (constant one)

1. $b * b = b$ | Enforces $b \in \{0,1\}$ (bitness)

2. $1 * 1 = b$ | Enforces $b = 1$

Range check

```
def main(private field secret){  
    assert(secret < 3);  
}
```

```
# curve:          bn128  
# constraint_count: 5  
def main(private _0) -> () {  
    # _1, _2 = Bits(2)((1 * ~one) * (1 * _0))  
    (1 * _1) * (1 * _1) == 1 * _1 // Bitness check failed  
    (1 * _2) * (1 * _2) == 1 * _2 // Bitness check failed  
    (1 * ~one) * (2 * _1 + 1 * _2) == 1 * _0 // Assertion failed at  
    (1 * ~one + (-1) * _1) * (1 * ~one + (-1) * _2) == 1 * _7  
    (1 * ~one) * (1 * ~one + (-1) * _1 + (-1) * _2 + (-1) * _7) == 0  
    return  
}
```

- $_0 \rightarrow x$ (private input)
- $_1 \rightarrow b1$ (bit)
- $_2 \rightarrow b2$ (bit)
- $\sim\text{one} \rightarrow 1$ (constant one)
- $_7 \rightarrow t$ (auxiliary)

1. $b1 * b1 = b1$ | Enforces $b1 \in \{0,1\}$
2. $b2 * b2 = b2$ | Enforces $b2 \in \{0,1\}$
3. $2*b1 + b2 = x$ | Reconstructs input x from bits ($x \in \{0,1,2,3\}$)
4. $(1 - b1)(1 - b2) = t$ | Defines auxiliary variable t ($b1=b2=1 \Rightarrow t=0$)
5. $1 - b1 - b2 - t = 0$ | Ensures consistency, excludes $b1=b2=1$



Setup



Setup

<https://zokrates.github.io/gettingstarted.html>



Demo: examples and ZoKrates pipeline



Compiling

```
zokrates compile -i xx.zok [-o out]
```

```
zokrates setup
```

```
zokrates compute-witness -a <arguments to main()>
```

```
zokrates generate-proof
```

```
zokrates verify
```

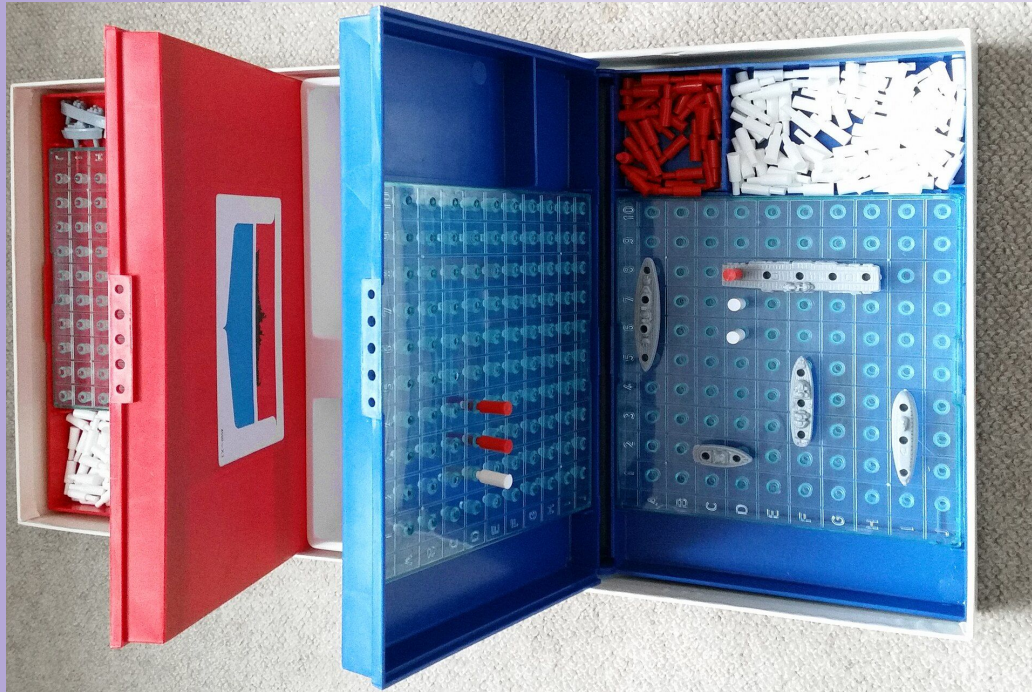
```
zokrates inspect --ztf
```




The battleships game



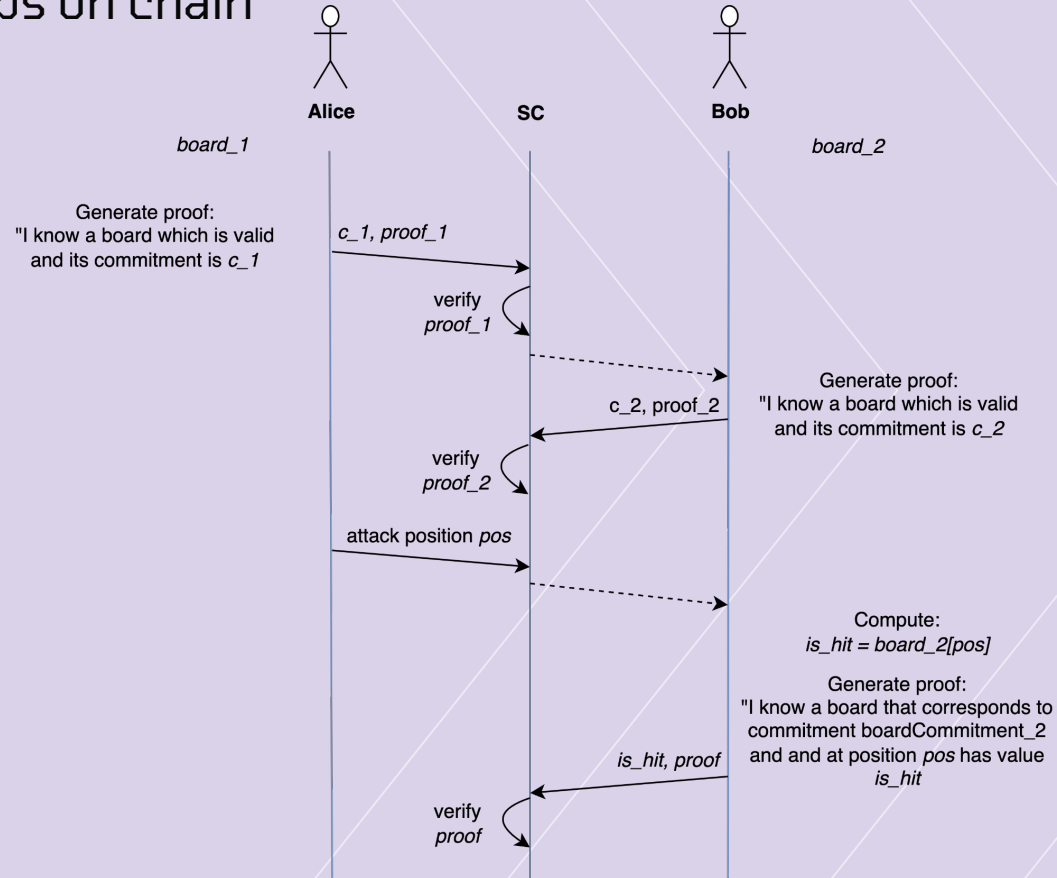
The battleships game



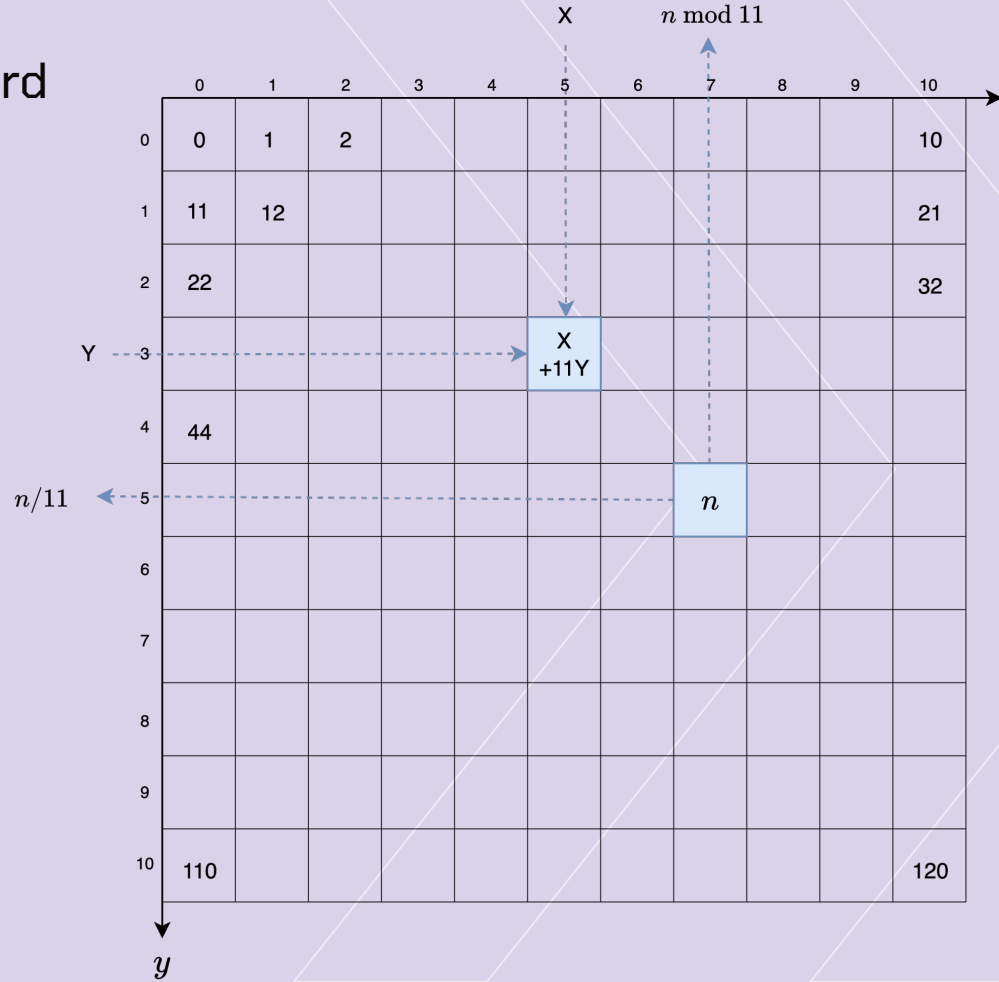
The battleships game on chain

- Hide the board
 - Can do with a vector commitment scheme
- Prove that the board is well-formed
 - Need to prove in zero knowledge!
 - Correct number and format of ships (in our example, 3 3x1 ships), ships don't touch
- Prove that an attack succeeded or failed

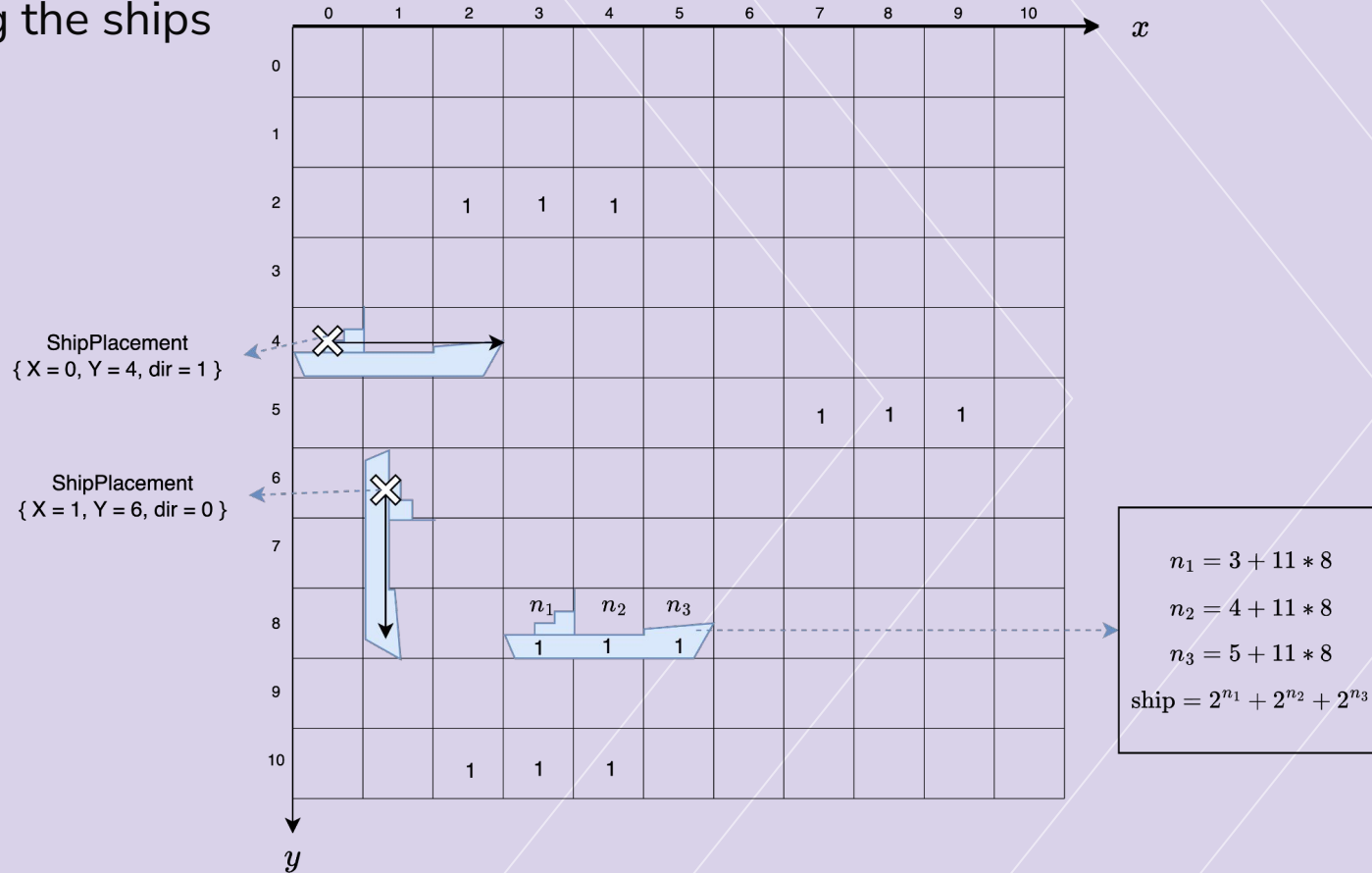
Playing battleships on chain



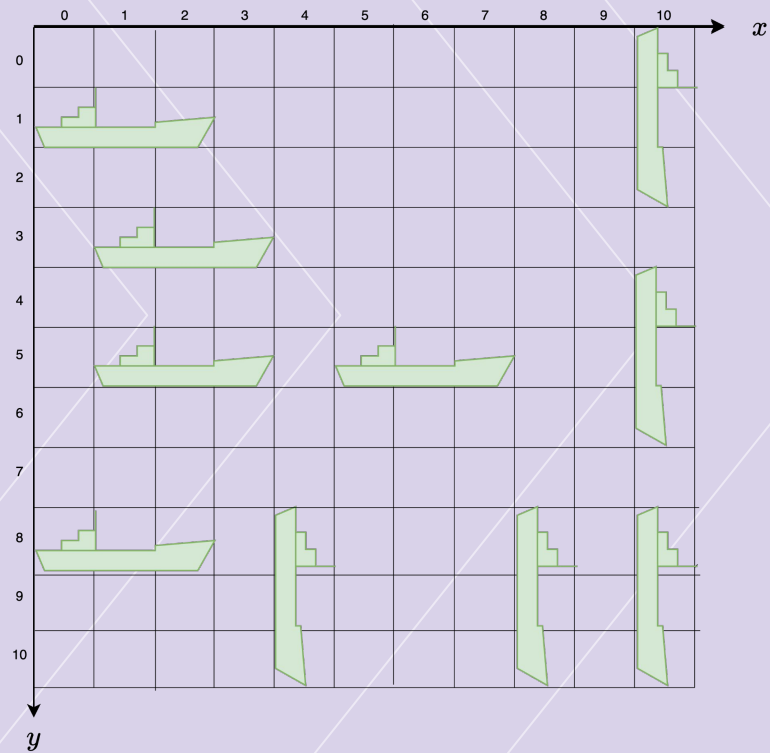
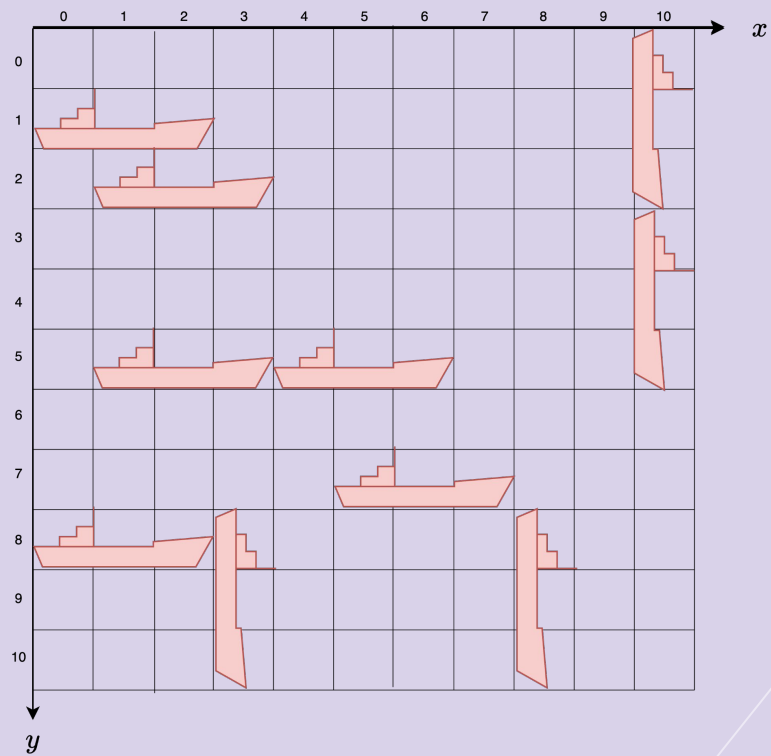
Indexing the board



Encoding the ships



The rules



The relations

“I know a set of `ship placements` of length 3 and a `randomness` such that `boardCommitment` commits to the board constructed by placing the 3x1 ships; the ships are all within the board and they don't touch each other (not even diagonally).”

“I know a `board` and a `randomness` such that `boardCommitment` is a commitment to `board` using randomness `randomness`, and that at position `position`, there is a ship if and only if `isHit` is true”



Demo: battleships on ZoKrates



Clone `https://github.com/commonprefix/zk-fleet` and checkout the *exercises branch*

Study `examples/*`

Implement TODOs in `board/main.zok`

Implement TODOs in `attack/main.zok`





Demo: using the prover and verifier



■ Demo: end 2 end demonstration

